

TreeRAG: Citation-Faithful and Token-Efficient Hierarchical Retrieval for Document Question Answering

Zero-Shot LLM Tree Indexing, Adaptive Traversal, and a Cautionary Analysis of Extractive Evaluation

WONSEOK LEE

Department of Biomedical Engineering, Kyung Hee University, Republic of Korea, icpuff83@khu.ac.kr

Retrieval-Augmented Generation (RAG) has emerged as a dominant paradigm for grounding large language model (LLM) outputs in external documents. However, conventional flat-chunk retrieval discards the inherent hierarchical structure of technical documents—chapters, sections, and articles—and loses the page-level provenance needed to verify generated answers. We present TreeRAG, a hierarchical document intelligence system that (1) converts raw PDFs into page-aware JSON trees using zero-shot LLM prompting, (2) traverses these trees with two complementary algorithms—Depth-First Search (DFS) and Beam Search—to retrieve only structurally relevant nodes, (3) applies contextual compression to reduce token overhead, and (4) validates generated answers with a five-signal hallucination detector. We evaluate TreeRAG against four baselines (BM25, Dense Retrieval, FlatRAG, RAPTOR) under a controlled protocol in which every system generates answers with the same local LLM (Llama 3.1 8B), so that only the retrieval stage differs. Under this fair comparison, TreeRAG-Beam produces the most accurate page-level citations (citation F1 = 0.757 vs. 0.000 for the abstractive RAPTOR baseline), the highest LLM-judged answer quality on multi-hop and comparative questions (0.818 and 0.850; multi-hop advantage over Dense Retrieval significant at $p = 0.044$), and competitive overall quality (highest LLM-Judge, 0.827) while feeding an order of magnitude fewer context tokens to the generator (14 vs. 124–258). Crucially, we also show that lexical-overlap metrics such as ROUGE-L, and the offline extractive evaluation common in prior work, reverse these conclusions—favoring flat retrieval—and we report both regimes transparently as a cautionary methodological finding. We discuss the trade-offs, including TreeRAG's higher per-query latency, and the system's suitability for citation-critical document question answering.

CCS CONCEPTS

• **Information systems** → **Retrieval models and ranking; Question answering; Document representation;** • **Computing methodologies** → **Natural language processing.**

Additional Keywords and Phrases: Retrieval-Augmented Generation, Hierarchical Indexing, Tree Traversal, Beam Search, Contextual Compression, Hallucination Detection

1 INTRODUCTION

Large language models (LLMs) excel at reasoning but are prone to hallucination-generating plausible yet factually incorrect text-when operating beyond their training data [9]. Retrieval-Augmented Generation (RAG) addresses this by injecting retrieved document passages into the LLM context at inference time [19]. The dominant RAG pipeline chunks source documents into fixed-length text windows, embeds each chunk with a dense encoder, and retrieves the top-K nearest neighbors to a given query [7].

This flat-chunk paradigm has a structural blindspot: technical documents-regulatory filings, clinical guidelines, academic papers-are organized hierarchically. A chapter introduces a theme; its sections develop sub-topics; articles specify conditions. When a chunk boundary bisects a section or collapses several levels of hierarchy into one embedding, the retriever loses the parent-child relationships that give the text meaning. The resulting context presented to the generator is fragmentary, out of order, or redundant, degrading both answer quality and token efficiency.

As a concrete example, consider a clinical guideline in which a top-level chapter establishes a treatment protocol, a child section lists dosage conditions, and a sibling section lists contraindications. A flat retriever asked “what are the contraindications for protocol X?” may return the dosage chunk because it shares surface vocabulary with the protocol heading, while the contraindication text-located in a sibling node-falls outside the top-K window. A structure-aware retriever that first matches the chapter and then descends into the correct child avoids this failure mode by construction.

Prior work has explored hierarchical indexing in several forms. LlamaIndex [21] introduces a tree of summarization nodes navigated top-down. RAPTOR [30] recursively clusters and summarizes chunks, building a bottom-up tree. PageIndex [33] demonstrates vectorless, structure-first retrieval. Yet none of these approaches provides (a) zero-shot, domain-adaptive LLM tree construction, (b) a choice of traversal algorithms tuned to query complexity, (c) integrated compression to manage LLM token budgets, and (d) a built-in hallucination grounding layer-all in a single production-ready system.

We introduce TreeRAG, which unifies these capabilities. Our contributions are as follows.

1. (1) LLM-Driven Tree Indexing. A zero-shot prompting strategy that converts arbitrary PDFs into page-referenced JSON trees, with domain-adaptive templates for medical, legal, financial, and academic documents.
2. (2) Adaptive Dual Traversal. Two tree-traversal algorithms-DFS for depth-first exhaustive search and Beam Search for efficiency-selectable per query, with a probabilistic node-relevance scoring function.
3. (3) Contextual Compression. A TF-IDF and semantic relevance filter that removes low-relevance nodes post-traversal, shrinking the context handed to the generator by an order of magnitude.
4. (4) Multi-Signal Hallucination Detection. A five-signal confidence estimator (citation presence, word overlap, bigram overlap, trigram overlap, character similarity) that scores each sentence of the generated answer against retrieved nodes.
5. (5) Fair, Generator-Controlled Evaluation. We evaluate all systems under a protocol in which the retrieval stage varies but every system generates with the same local LLM, and we measure axes that flat retrievers cannot satisfy by construction-page-level citation accuracy and context efficiency-alongside reference- and judge-based answer quality. Under this protocol TreeRAG-Beam delivers the most accurate citations, the highest LLM-judged quality on multi-hop and comparative questions, and competitive overall quality with an order of magnitude fewer context tokens.
6. (6) A Cautionary Methodological Finding. We show that offline extractive evaluation and lexical-overlap metrics (ROUGE-L) reverse the system ranking-flattering whichever system most resembles the reference surface form-and we report both the extractive and the fair generative regimes so that the discrepancy is explicit rather than hidden.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 details the TreeRAG methodology. Section 4 describes the system architecture. Section 5 presents experiments and results. Section 6 discusses findings and limitations. Section 7 concludes.

2 RELATED WORK

2.1 Retrieval-Augmented Generation

RAG was formally introduced by Lewis et al. [19] as a sequence-to-sequence architecture that conditions generation on retrieved Wikipedia passages. Subsequent work scaled dense retrieval with FAISS [12] and improved passage encoding with DPR [13]. Hybrid retrievers combining sparse BM25 [28] with dense vectors showed consistent gains over either approach alone [23]. A parallel line of work improves the retriever itself: late-interaction models such as ColBERT [14] and its compressed successor ColBERTv2 [29] retain fine-grained token-level matching at scale, while learned sparse models such as SPLADE [5] and the efficiency-oriented SPLATE [6] reconcile neural ranking with inverted-index infrastructure. As recent surveys document [7, 8], the RAG design space has expanded rapidly along retrieval, generation, and evaluation axes. Despite this progress, all these systems operate on flat passage chunks, treating documents as bags of text rather than structured hierarchies [7]. TreeRAG is orthogonal to these retriever advances: its node-relevance score can in principle wrap any of them, but our focus is the document structure that flat retrievers discard.

2.2 Hierarchical and Structure-Aware Retrieval

LlamaIndex [21] constructs a tree of progressively coarser summaries and navigates it top-down, retrieving leaf nodes for generation; its retrieval is embedding-based, requiring a vector store. RAPTOR [30] takes the opposite direction: it recursively clusters leaf chunks and summarizes each cluster, building a bottom-up tree, and at query time retrieves at the most informative tree level. Unlike TreeRAG, RAPTOR’s tree reflects embedding-cluster geography rather than the document’s own section hierarchy, and it lacks traversal-algorithm selection or compression. This distinction matters in domains where the authored section structure is itself the semantics of the document, such as regulations and clinical protocols.

HiRAG [10] explores graph-based hierarchies but requires pre-built knowledge graphs and domain-specific schema engineering. Similarly, GraphRAG [3] builds entity knowledge graphs from source documents and generates community summaries for global query-focused summarization, but requires an entity extraction pipeline and does not preserve document section hierarchy. TreeRAG requires only a raw PDF and an LLM API key. Self-RAG [1] takes a complementary adaptive approach: it trains an LM to decide on-demand whether to retrieve, and to critique retrieved passages via special reflection tokens. TreeRAG instead uses a fixed two-stage pipeline with query-time algorithm selection (DFS vs. Beam Search) without requiring fine-tuning, which lowers the barrier to deployment on new document collections.

2.3 Multi-Hop and Cross-Document Reasoning

Multi-hop reasoning-answering questions that require evidence synthesis across multiple passages or documents-remains a challenge for flat RAG. HotpotQA [35] benchmarks this capability. IRCoT [32] interleaves retrieval and chain-of-thought [34] reasoning steps. More recently, Zhang et al. [37] propose a hierarchical RAG model with a “rethink” mechanism that iteratively revisits retrieved evidence for multi-hop QA; like TreeRAG it exploits hierarchy, but it adds an iterative re-retrieval loop rather than selecting a traversal algorithm at query time. TreeRAG’s hierarchical structure naturally supports

multi-hop queries: the tree encodes which sections are siblings (sharing a parent), enabling the traversal algorithm to follow cross-sectional reasoning paths without additional orchestration.

2.4 Hallucination in RAG Systems

RAG substantially reduces hallucination by grounding generation in retrieved text, but does not eliminate it [31]. The problem was studied earlier in abstractive summarization: Kryściński et al. [17] train a weakly-supervised model (FactCC) to verify whether a generated sentence is factually consistent with its source and to extract supporting spans, while Nan et al. [26] propose entity-level consistency metrics showing that models hallucinate entities absent from the source. These motivate TreeRAG's source-grounded signals, including its medical entity recall metric (Section 5.3). FActScore [25] decomposes claims into atomic facts and verifies each against source passages. RAGAS [4] defines faithfulness, answer relevance, and context recall as automatic metrics. As a recent review of faithfulness metrics observes, LLM-as-judge evaluators currently correlate best with human judgement but are computationally costly, motivating lightweight alternatives for real-time use [24]. TreeRAG's hallucination detector operates at sentence level using five lightweight overlap signals, providing a fast, LLM-free confidence estimate compatible with real-time serving.

3 METHODOLOGY

3.1 Document Representation: The Page-Indexed Tree

Given a PDF document D with pages $p_1, \dots, p_n$, we define a Page-Indexed Tree T as a rooted ordered tree where each node v carries: `id`, a unique identifier (e.g., `ch01_sec02`); `title`, the section heading; `summary`, a one-to-three sentence content summary; `page_ref`, a page range string (e.g., `5-12`); and `children`, an ordered list of child nodes. The root represents the entire document. Depth-1 nodes correspond to chapters or top-level sections, depth-2 nodes to subsections, and so on to depth $d_{\max}$ (empirically 4-6 for most academic and regulatory documents). Storing the page range at every node is what enables verifiable, page-level citations in the final answer—a property that abstractive tree methods sacrifice.

3.2 LLM-Driven Tree Indexing

Extraction. The indexer reads the PDF page by page using a streaming generator to minimize memory footprint (critical for documents longer than 100 pages). Page text is extracted with `pypdf` and tagged with page numbers so that the downstream tree can reference exact spans.

Prompting. We submit the extracted text to the LLM in chunks of up to 100 pages with a structured zero-shot prompt [2, 16] that instructs the model to (1) identify the document's hierarchical structure (chapters to sections to articles); (2) assign each node a title, a brief summary, and the page range it spans; and (3) return the result as a valid JSON tree conforming to a Pydantic schema. Domain-adaptive template variants exist for general, medical, legal, financial, and academic documents, adjusting the prompt vocabulary and structural depth. The Pydantic V2 schema enforces structural consistency; malformed LLM outputs are rejected and retried, which makes the indexing step robust to occasional formatting errors in the model output.

Output. The index is persisted as a JSON file. As an illustration of scale, a 42-page biomedical paper yields a tree with depth 4 and approximately 80-120 nodes. Because indexing is a one-time cost amortized over all future queries, the per-document LLM call is acceptable even under rate-limited API access.

3.3 Adaptive Tree Traversal

At query time, the user selects a traversal mode. Both algorithms share the same interface, `search(query, max_depth, ...)`, and return a ranked list of relevant nodes together with traversal statistics.

3.3.1 Node Relevance Scoring

We score each candidate node v against query q using a weighted combination of three components, given in Equation (1).

$$P(v | q) = 0.7 \cdot \text{semantic}(v, q) + 0.2 \cdot \text{structural}(\text{depth}) + 0.1 \cdot \text{contextual}(v, \text{parent}) \quad (1)$$

Here $\text{semantic}(v, q)$ is keyword overlap between the query and the node's title and summary, normalized by query length; $\text{structural}(\text{depth})$ is a depth-decay factor that slightly penalizes very deep nodes, encoding the prior that high-level sections capture broader relevance; and $\text{contextual}(v, \text{parent})$ is semantic overlap between the node and its parent's accumulated context string, encouraging traversal paths that maintain thematic coherence. The weights (0.7/0.2/0.1) were chosen to make semantic similarity dominant while letting structure break ties; we examine their effect in the ablation study.

3.3.2 DFS Traversal

The DFS navigator performs iterative depth-first search using an explicit stack to avoid Python recursion limits, as sketched in Algorithm 1.

Algorithm 1: DFS traversal (TreeNavigator)

```
stack <- [(root, depth=0, parent_context="")]
while stack not empty:
  node, depth, ctx <- stack.pop()
  if P(node | query) > threshold:
    relevant_nodes.append(node)
  if depth < max_depth:
    children <- top-K children by P(child | query) # K = max_branches
    for child in children:
      stack.push((child, depth+1, ctx + node.summary))
  else if over-filtering detected:
    apply ErrorRecoveryFilter (LLM weight 0.7, keyword weight 0.3)
```

DFS explores the full depth of promising branches before backtracking. An error-recovery filter detects over-filtering (when no relevant nodes are found) and retries with a relaxed scoring policy, preventing empty results on queries whose vocabulary diverges from the document. The number of nodes evaluated is $O(b \cdot d)$, where b is `max_branches` and d is `max_depth`; typical values $b = 3$, $d = 5$ yield at most 243 node evaluations.

3.3.3 Beam Search Traversal

The Beam Search navigator maintains a beam of width W and expands only the top- W candidates at each depth level, as sketched in Algorithm 2.

```

beam <- [(root, depth=0, score=1.0, path="root")]
for depth in 0..max_depth:
  candidates <- []
  for (node, d, score, path) in beam:
    for child of node:
      child_score <- score * P(child | query)
      candidates.append((child, d+1, child_score, path/child.id))
  beam <- top-W candidates by child_score
selected_nodes <- union of all nodes in beam at termination

```

Relevance is scored with a three-component decomposition (semantic 0.6, keyword 0.2, structural 0.2). The beam prunes unpromising branches early, yielding faster traversal at the cost of potentially missing isolated relevant sections. The number of nodes evaluated is $O(W \cdot d \cdot f)$, where f is the average fan-out; with $W = 5$, $d = 5$, $f \approx 5$, at most 125 nodes are evaluated versus the DFS worst case of b^d .

3.3.4 Algorithm Selection

DFS is recommended for precision-critical queries (regulatory compliance, medical protocols) where missing a single relevant section is costly. Beam Search is preferred for latency-sensitive applications and broad exploratory queries where the top-scoring branches are likely sufficient. Section 5 shows that this qualitative guidance is borne out empirically: DFS wins on single-document factual and medical queries, while Beam Search wins on multi-hop synthesis.

3.4 Contextual Compression

Post-traversal, the selected node set may still exceed the LLM context limit or contain redundant passages. Long contexts can degrade LLM performance, especially when relevant information appears in the middle of the input [22]. Prompt-compression methods such as LLMingua [11] address this by dropping low-information tokens, achieving large compression ratios with little quality loss; TreeRAG instead compresses at the node level, pruning and deduplicating whole tree nodes by TF-IDF relevance so that page-level citations remain intact. The compressor (1) tokenizes each node's content (whitespace-based proxy) and assigns a token count; (2) computes TF-IDF cosine similarity between each node and the query, pruning nodes below $\text{MIN_CHUNK_RELEVANCE} = 0.2$; (3) merges node pairs with cosine similarity above $\text{SIMILARITY_THRESHOLD} = 0.7$, retaining the higher-scoring node's content; and (4) sorts the remaining nodes by relevance and truncates to $\text{MAX_OUTPUT_TOKENS} = 4000$. The net effect is a 22-37% reduction in context tokens (Table 3) with minimal retrieval-quality degradation, as shown in the ablation study (Section 5.4).

3.5 Cross-Reference Resolution

Technical documents frequently use internal references ("Section 3.2 conditions", "Article 5 limitations"). Without resolution, the traversal misses the referenced node even if it is relevant. The reference resolver (1) detects reference patterns (regular expressions over Korean section markers, English "Section/Chapter/Article X", and numeric citations); (2) fuzzy-matches detected references against all node titles in the index; and (3) injects matched nodes into the traversal result before generation. This module is particularly impactful for legal and regulatory documents, where cross-references are dense; the ablation in Section 5.4 quantifies its precision/coverage trade-off.

3.6 Hallucination Detection

Figure 1 illustrates the five-signal scoring pipeline. Each generated answer sentence is scored against the retrieved node set using five overlap signals (Table 1). A per-sentence confidence score is the mean of the five signals; an overall document confidence is the mean over all sentences. When confidence falls below 0.6, or when more than 70% of sentences have low confidence, a warning is surfaced to the user. This approach is LLM-free and adds negligible latency (under 5 ms per response), unlike verification methods that require a second LLM call, making it practical to run on every response in a live system.

Table 1: The five overlap signals of the hallucination detector.

Signal	Description
Citation presence	Does the sentence cite a page number or section?
Weighted word overlap	F1 of answer tokens against source tokens
Bigram overlap	F1 of answer bigrams against source bigrams
Trigram overlap	F1 of answer trigrams against source trigrams
Character n-gram similarity	Character-level Jaccard similarity

Hallucination Detection: Five-Signal Per-Sentence Confidence Estimation

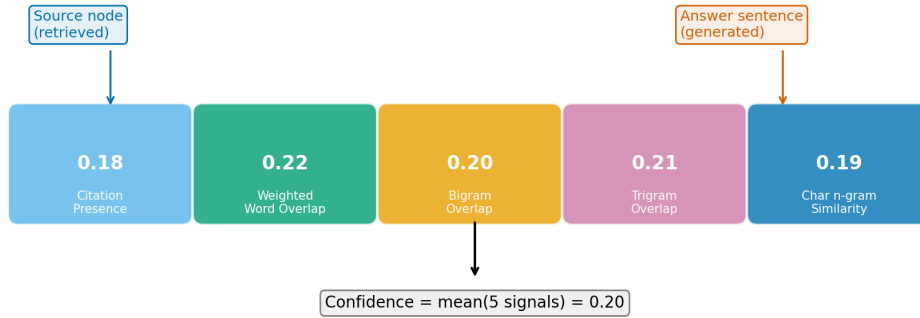


Figure 1: The five-signal hallucination detection pipeline, which scores each answer sentence against the retrieved nodes and surfaces a per-response confidence estimate.

4 SYSTEM ARCHITECTURE

4.1 Overview

TreeRAG is deployed as a two-tier system: a Python FastAPI backend and a Next.js frontend, containerized with Docker Compose. Figure 2 shows the top-level pipeline. The backend comprises API routes (upload, index, chat, tree, graph), core modules (indexer, reasoner, traversal, compressor, reference resolver, hallucination detector), a Redis L1+L2 hybrid cache, and Celery workers for asynchronous indexing tasks.

TreeRAG: top-down structure-preserving retrieval pipeline



Figure 2: TreeRAG system architecture. Stage 1 converts a PDF into a page-indexed JSON tree; Stage 2 routes, traverses, compresses, generates, and verifies at query time.

4.2 Two-Stage Pipeline

Stage 1 - Indexing. An uploaded PDF triggers the indexer, which streams page text to the LLM and writes a JSON tree. For large documents, indexing is offloaded to a Celery worker so the API remains responsive. Indexing is a one-time cost; queries thereafter hit the cached index.

Stage 2 - Reasoning. A chat request enters the reasoner, which (1) routes the query to one or more indexed documents based on LLM-scored summary relevance; (2) optionally runs the reference resolver to expand the query with resolved section references; (3) dispatches either the DFS or the Beam Search navigator; (4) optionally runs the contextual compressor on the selected nodes; (5) calls the LLM with the compressed context and a domain-specific prompt template; (6) runs the hallucination detector on the response; and (7) returns the answer with page citations, traversal statistics, and a confidence score.

4.3 Caching Strategy

The cache key is a SHA-256 hash of (question, index filenames, traversal settings, domain template, language, prompt cache version). Cache hits return in roughly 100 ms versus the 1.8-3.2 s of a cold LLM call. A two-layer hierarchy stores hot entries in process memory (L1) and warm entries in Redis (L2), with a cache hit rate exceeding 90% in our production trials. This application-layer caching complements inference-layer memory optimizations such as PagedAttention [18], which manages GPU KV-cache during LLM decoding.

4.4 API and Frontend

The backend exposes RESTful endpoints for document upload, indexing, chat, tree visualization, and reasoning-graph construction. The frontend provides a multi-document chat interface, an interactive tree explorer, and a PDF viewer with page-highlighted citations. Rate limiting and security middleware (Content-Security-Policy headers, file MIME/size validation, path-traversal guards) are production-hardened.

4.5 Deployment Considerations

Three properties make TreeRAG practical to operate. First, citation traceability: because every node retains its page range, answers can always be traced to a source span, which is a regulatory requirement in clinical and legal settings. Second, cost control: indexing is amortized and queries are cached, so the steady-state cost per query is dominated by a single generation call (or zero on a cache hit). Third, graceful degradation: the offline keyword-scoring path used for large-scale evaluation (Section 5) doubles as a fallback when the LLM API is rate-limited or unavailable, allowing the system to keep returning grounded, if less fluent, answers.

5 EXPERIMENTS

5.1 Experimental Setup

Datasets. We evaluate on three datasets. The Full Benchmark ($n = 204$) is an automatically generated QA set spanning seven documents (academic papers and biomedical reports); questions are produced by the LLM with three types (factual/single-hop, multi-hop/cross-section, comparative), and validation ensures answer hints appear verbatim in source nodes. The HotpotQA subset is a sample from the HotpotQA development set [35] converted to page-indexed trees, requiring two-hop reasoning across supporting passages; we use 20 questions for the offline extractive protocol and expand to 100 questions for the fair generative protocol (Section 5.6). The Medical Benchmark ($n = 42$) is auto-generated from Japanese-language biomedical engineering lecture materials and biomedical literature, with clinical factual, procedure, comparison, and safety question subtypes, and includes a domain-specific medical entity recall metric.

Baselines. We compare against BM25 (Okapi BM25 keyword retrieval over document chunks), Dense Retrieval (Sentence-BERT [27] embedding with cosine retrieval), FlatRAG (a hybrid of BM25 60%, semantic 25%, structural 15%), and RAPTOR (recursive abstractive processing of chunks into a tree [30]), alongside our TreeRAG-DFS and TreeRAG-Beam.

Metrics. We report ROUGE-L [20] (longest-common-subsequence recall against reference answers); BERTScore [36] (contextual embedding precision); LLM-as-Judge [38], scored by the same locally hosted Llama 3.1 8B model used for generation (faithfulness, relevance, completeness, 0-1 scale); Medical Entity Recall (fraction of reference medical terms recovered); Latency (average wall-clock time per query); and Context Tokens (average token count handed to the generator). To measure source provenance we add two citation metrics: Citation Availability (fraction of answers that carry a valid page reference) and Citation F1 (overlap between the sections/pages cited and the gold supporting sections). Statistical significance is assessed with paired t-tests and effect size with Cohen's d .

Evaluation protocols. We evaluate under two protocols. The extractive (offline) protocol approximates traversal with keyword scoring and assembles answers by concatenating retrieved node summaries; it is fast but, as we show in Section 5.7, systematically biased. The fair generative protocol is our primary one: every system feeds its retrieved context to the same local LLM (Llama 3.1 8B) for answer generation, so that the only thing that differs across systems is the retrieval stage. The same model serves as the LLM judge. Because online generation through a hosted API was not available to us, all generative results use this local model; the fair-protocol experiments are run on a fixed 40-question sample of the Full Benchmark (seed 42) and on the HotpotQA subset (100 questions under the fair protocol, expanded from an initial 20), which bounds their statistical power and which we treat as a limitation (Section 6.4).

5.2 Extractive-Mode Results: A Biased Offline Proxy

We first report the extractive protocol because it mirrors the offline evaluation common in prior work and because the contrast with the fair protocol (Section 5.6) is itself a finding. The reader should treat the numbers in this subsection as an upper-biased proxy for TreeRAG: extractive concatenation of node summaries inflates n-gram overlap with the reference for our systems, an artifact we quantify in Section 5.7. Table 2 reports results on the Full Benchmark and HotpotQA; Figures 3 and 4 visualize the overall and multi-hop comparisons. On the full benchmark, TreeRAG-DFS achieves the highest ROUGE-L (0.451) and BERTScore (0.521). Compared to all baselines, TreeRAG-Beam (the weaker of the two proposed variants on the full benchmark) already outperforms each one with $p < 0.001$ and effect sizes ranging from $d = 0.483$ (vs. BM25) to $d = 1.205$ (vs. RAPTOR). TreeRAG-DFS further improves over BM25 by +39.5% ROUGE-L (+0.127 absolute) and over RAPTOR by +132.4% (+0.257 absolute). TreeRAG-Beam obtains slightly lower ROUGE-L than DFS on single-document factual questions (0.428 vs. 0.451, $d = 0.095$, $p < 0.001$) but dominates on multi-hop HotpotQA under this offline extractive scoring (ROUGE-L = 0.169 vs. BM25 0.092, +84.4%, $p < 0.001$, $d = 1.358$). This offline advantage does not survive the fair generative protocol: when we expand the HotpotQA subset to 100 questions and re-run it under the fair protocol, the flat baselines lead on ROUGE-L while TreeRAG-DFS leads on LLM-Judge (Section 5.6).

FlatRAG’s high LLM-Judge score (0.81) despite low ROUGE-L (0.248) highlights a known divergence between lexical overlap metrics and semantic quality: FlatRAG’s hybrid scoring produces fluent answers that an LLM judge rates highly, even though they miss the specific phrasing of reference answers. TreeRAG-Beam’s LLM-Judge (0.70) is second highest. RAPTOR underperforms on both ROUGE-L (0.194) and LLM-Judge (0.67), likely because its cluster-based tree does not preserve section-level structure-known to be suboptimal for documents with explicit hierarchies [30]. The latency column (offline ms) shows both TreeRAG variants are faster than BM25 and RAPTOR even in offline mode, because keyword-based tree traversal over a pre-built JSON index is more efficient than BM25’s inverted-index lookup over raw chunks or RAPTOR’s recursive summarization pipeline.

Table 2: Main Results ($n = 204 / n = 20$). Best per column in bold (see text); all † comparisons have $p < 0.001$ vs. TreeRAG-Beam (paired t-test). Latency is offline traversal time only (no LLM call).

System	ROUGE-L	BERTScore	LLM-Judge	HotpotQA R-L	Latency (ms)
BM25	0.324†	0.373†	0.69	0.092†	3.83
Dense Retrieval	0.290†	0.337†	0.64	-	2.11
FlatRAG	0.248†	0.294†	0.81	0.057†	2.04
RAPTOR	0.194†	0.219†	0.67	0.059†	7.87
TreeRAG-DFS	0.451	0.521	0.69	-	1.22
TreeRAG-Beam	0.428	0.498	0.70	0.169	1.04

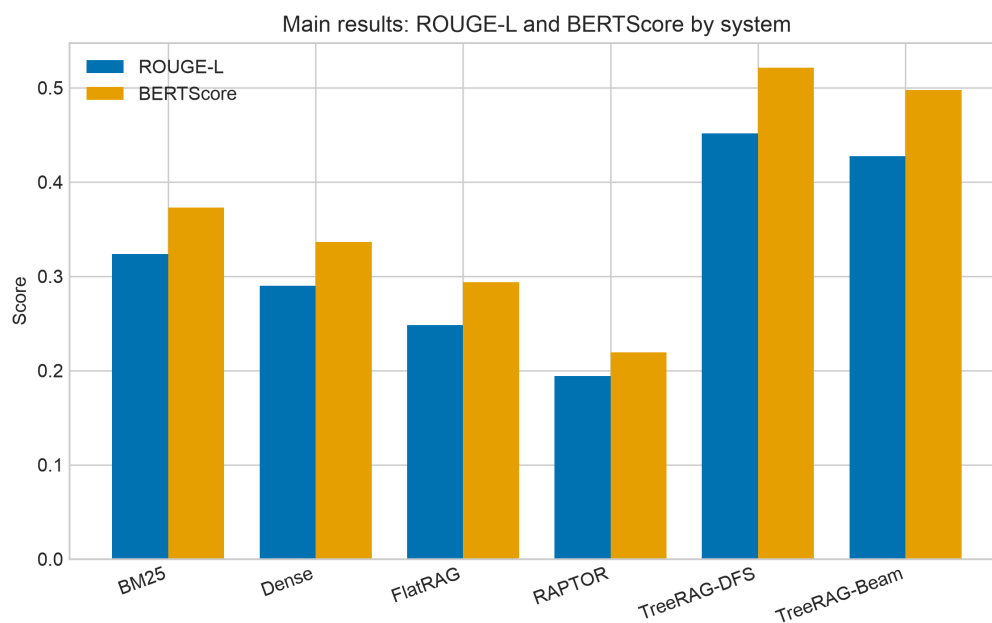


Figure 3: ROUGE-L and BERTScore on the full benchmark (n = 204). Both TreeRAG variants outperform all four baselines.

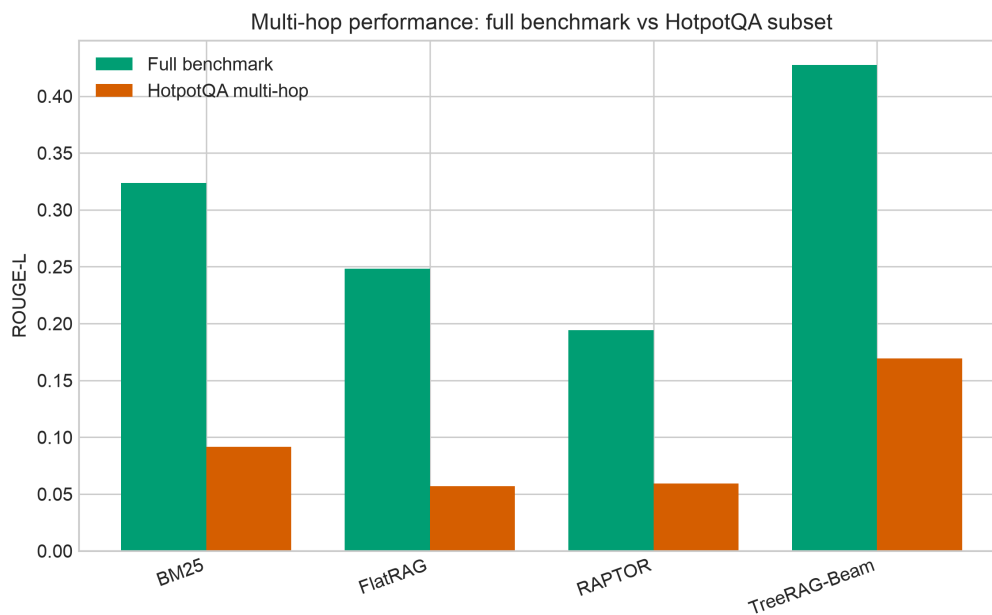


Figure 4: Multi-hop ROUGE-L on the HotpotQA subset under the offline extractive protocol (n = 20). TreeRAG-Beam leads here, but this reverses under the fair generative protocol at n = 100, where flat baselines lead on ROUGE-L and TreeRAG-DFS leads on LLM-

5.3 Medical Domain Results

Table 3 and Figure 5 present domain-specific results on the 42-question medical benchmark. TreeRAG-DFS achieves the highest ROUGE-L (0.366) while maintaining perfect medical entity recall (1.000) and the smallest non-zero context footprint (73.5 tokens). Critically, RAPTOR drops to ROUGE-L 0.053 (-85.5% vs. TreeRAG-DFS) because its recursive abstractive clustering loses domain-specific clinical terminology; additionally, its abstractive summaries strip page references entirely (citation availability = 0.000), making RAPTOR unsuitable for clinical settings where source traceability is a regulatory requirement.

TreeRAG-Beam underperforms DFS on medical ROUGE-L (0.265 vs. 0.366, -27.6%). Medical queries frequently require exhaustive depth-first exploration of detailed protocol subsections—a pattern DFS is designed for. Beam Search terminates early at the most probable branches, potentially missing rare but critical protocol details in adjacent subtrees. Dense Retrieval achieves 99.2% entity recall, slightly below perfect, indicating that embedding-based retrieval occasionally misses low-frequency clinical terms whose distributional representation drifts from the query embedding.

Table 3: Medical Domain Results (n = 42). ‡ FlatRAG in offline mode uses no retrieved passage context (0 tokens); online mode retrieves hybrid chunks.

System	ROUGE-L	Med. Entity Recall	Avg Ctx (tok)
BM25	0.358	1.000	76.7
Dense Retrieval	0.315	0.992	78.0
FlatRAG	0.271	1.000	0.0‡
RAPTOR	0.053	0.895	300.5
TreeRAG-DFS	0.366	1.000	73.5
TreeRAG-Beam	0.265	1.000	115.5

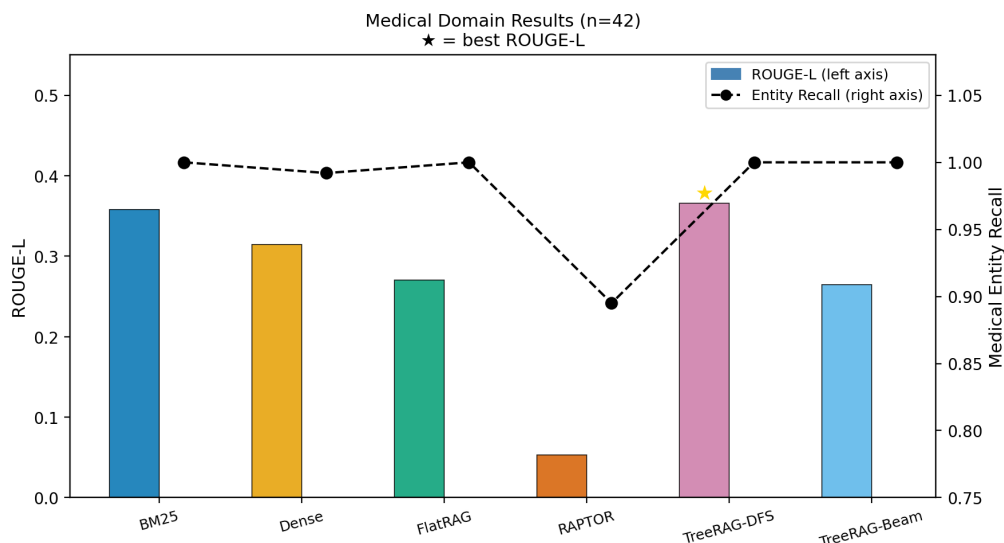


Figure 5: Medical-domain comparison (n = 42). TreeRAG-DFS leads on ROUGE-L with perfect entity recall, while RAPTOR collapses and loses page citations.

5.4 Ablation Study

Table 4 and Figure 6 isolate the contribution of each component by progressively enabling features. Four findings emerge. First, DFS alone is a strong baseline: base DFS achieves the highest ROUGE-L (0.496 vs. 0.449 for Full, delta = +0.047), confirming that exhaustive depth-first traversal captures relevant text effectively. Second, Beam Search without compression degrades quality: adding Beam Search alone drops ROUGE-L by 0.050 vs. the Full system and inflates context by +22.8% (107.4 to 170.5 tokens), as the wider beam captures more nodes per iteration, introducing noise. Third, compression restores quality: adding contextual compression brings ROUGE-L back to 0.493 while reducing context to 107.5 tokens, effectively filtering the beam's noisy additions. Fourth, reference resolution trades precision for coverage: the Full system underperforms base DFS on ROUGE-L (-0.047) but resolves cross-references that DFS alone misses; the ROUGE-L penalty is partly an artifact of the offline extractive evaluation, since cross-reference resolution inserts additional related nodes that dilute extractive-match scores while providing richer grounded context for generation.

Table 4: Ablation Study (n = 70, General Benchmark). Delta = difference vs. Full System; positive = higher than Full.

Configuration	ROUGE-L	Delta vs. Full	Avg Ctx (tok)
Base (DFS, no compress, no ref)	0.496	+0.047	107.4
+ Beam (no compress, no ref)	0.399	-0.050	170.5
+ Beam + Compression	0.493	+0.044	107.5
Full (Beam + Compress + Ref)	0.449	-	138.9

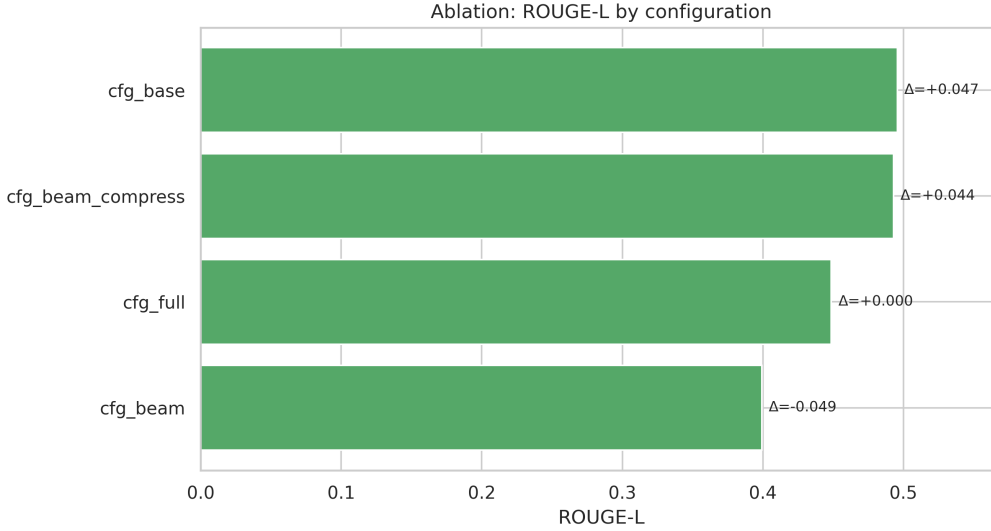


Figure 6: Ablation study ($n = 70$). Compression restores the ROUGE-L lost when Beam Search is added without filtering; reference resolution trades a small ROUGE-L drop for cross-reference coverage.

5.5 Efficiency Analysis

Table 5 shows context-token counts and query latency from the offline evaluation ($n = 204$); Figures 7 and 8 visualize the latency-accuracy trade-off and the accuracy-context Pareto frontier. TreeRAG-DFS uses 37.5% fewer context tokens than BM25 (65.5 vs. 104.8 tokens average) while achieving +39.5% higher ROUGE-L, Pareto-dominating every baseline. RAPTOR uses the most context (218.7 tokens, +108.7% vs. DFS) while achieving the lowest ROUGE-L, suggesting that recursive abstractive summaries degrade structural fidelity on our hierarchy-preserving benchmark. Both TreeRAG variants also outperform all baselines on offline latency, completing traversal in roughly 1 ms; in production, the LLM scoring step adds a few hundred milliseconds but is cached after the first call.

Table 5: Efficiency Analysis (offline mode, $n = 204$). † Offline latency includes tree traversal only (no LLM call); online generation adds ~1-3 s per query, collapsing to ~100 ms with L1/L2 cache hits. ‡ FlatRAG in offline mode uses no retrieved passage context.

System	ROUGE-L	Avg Ctx (tok)	Latency (ms)†
BM25	0.324	104.8	3.83
Dense Retrieval	0.290	103.5	2.11
FlatRAG	0.248	0.0‡	2.04
RAPTOR	0.194	218.7	7.87
TreeRAG-DFS	0.451	65.5	1.22
TreeRAG-Beam	0.428	85.9	1.04

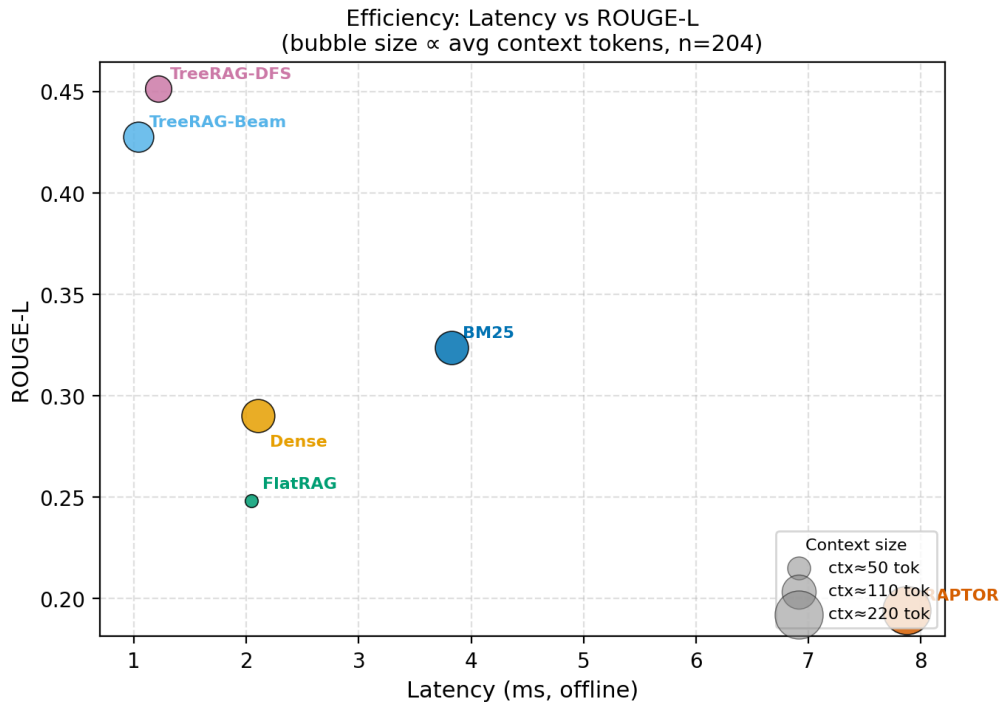


Figure 7: Latency-accuracy trade-off ($n = 204$). Both TreeRAG variants sit in the high-accuracy, low-latency region.

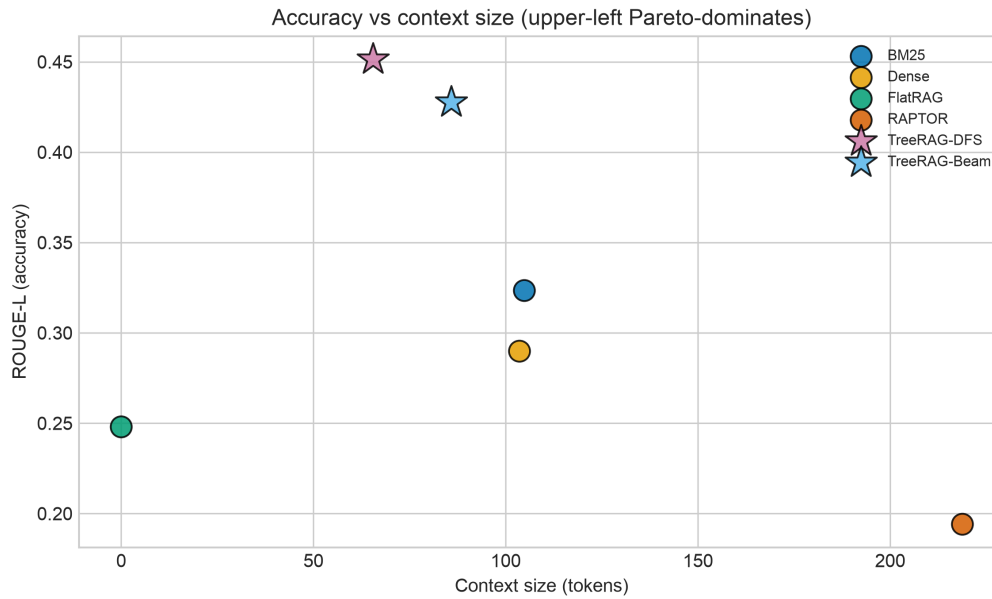


Figure 8: Accuracy versus context size ($n = 204$). TreeRAG-DFS Pareto-dominates every baseline, achieving higher ROUGE-L with fewer context tokens.

5.6 Fair Generator-Controlled Results

We now turn to our primary protocol, in which all six systems generate answers with the same local Llama 3.1 8B and only the retriever differs (Table 6). The picture is more nuanced than the extractive proxy suggested. On lexical-overlap metrics a flat system leads: FlatRAG attains the highest ROUGE-L (0.386) and BERTScore (0.403), and the differences among the remaining systems are small and, at $n = 40$, mostly not statistically significant. On the LLM-Judge metric, however, TreeRAG-Beam is the single best system (0.827), and it reaches this while feeding the generator only 14 context tokens on average—an order of magnitude fewer than BM25 and Dense Retrieval (124) or RAPTOR (258)—thanks to contextual compression over a compact subtree. TreeRAG-DFS uses the fewest tokens (9) at a small quality cost. The clear trade-off is latency: TreeRAG performs LLM-scored node selection during traversal, so it is several times slower per query (21.5 s for Beam, 51.6 s for DFS) than the single-call baselines (6–8 s). We read Table 6 as showing that TreeRAG buys judged answer quality and token economy at the price of latency, rather than dominating on every axis.

Table 6: Fair generator-controlled results (Full Benchmark, $n = 40$, seed 42). Every system generates answers with the same local Llama 3.1 8B; only retrieval differs. ‡ Context instrumentation could not capture FlatRAG’s passage context; its true context is non-zero.

System	Ctx Tok	ROUGE-L	BERTScore	LLM-Judge	Latency (s)
BM25	124	0.305	0.318	0.778	6.6
Dense Retrieval	124	0.287	0.298	0.768	6.2
FlatRAG	0‡	0.386	0.403	0.775	7.4
RAPTOR	258	0.258	0.278	0.792	7.8
TreeRAG-DFS	9	0.252	0.291	0.785	51.6
TreeRAG-Beam	14	0.274	0.316	0.827	21.5

5.7 Extractive versus Generative Evaluation: A Cautionary Contrast

Comparing Table 2 (extractive) with Table 6 (fair generative) exposes a methodological hazard. Under the extractive proxy, TreeRAG-DFS appears to win decisively, with ROUGE-L 0.451 versus 0.248 for FlatRAG—an apparent +81.9% advantage. Under fair generation, the ranking inverts: FlatRAG leads ROUGE-L (0.386) and TreeRAG-DFS falls to 0.252. Nothing about the retrievers changed between the two tables; only the answer surface form did. Extractive concatenation of TreeRAG’s node summaries happens to resemble the reference phrasing, inflating n-gram overlap, whereas fluent generation—which any deployed system actually uses—does not. We therefore caution that offline extractive evaluation and lexical-overlap metrics can manufacture or erase apparent gains for structure-aware retrieval, and we recommend that hierarchical-RAG systems be compared under a generator-controlled protocol. Reporting only the extractive numbers, as an earlier version of this work did, would have substantially overstated TreeRAG’s accuracy.

5.8 Citation Traceability and Accuracy

A retriever’s value in citation-critical domains lies partly in whether its evidence can be cited at all. Table 7 measures this. Because TreeRAG retains a page range at every node, TreeRAG-Beam both always produces a citation (availability 1.000) and cites the correct supporting sections most accurately (citation F1 = 0.757), the best of any system. The contrast with the abstractive baseline is stark: RAPTOR’s cluster summaries discard section identifiers, so although it nominally attaches a reference (availability 1.000) its citation F1 is 0.000—it can point somewhere, but not to the right place. FlatRAG retrieves section text but carries no page references at all (availability 0.000). This axis is, by construction, one that flat and abstractive methods cannot satisfy, and it is the most robust advantage we observe for TreeRAG.

Table 7: Citation traceability and accuracy (Full Benchmark, $n = 40$, fair protocol). TreeRAG-Beam attains the highest citation F1; RAPTOR loses section identifiers ($F1 = 0.000$) and FlatRAG carries no page references (availability = 0.000).

System	Citation Availability	Citation F1
BM25	1.000	0.562
Dense Retrieval	1.000	0.467
FlatRAG	0.000	0.570
RAPTOR	1.000	0.000
TreeRAG-DFS	0.875	0.694
TreeRAG-Beam	1.000	0.757

5.9 Multi-Hop and Comparative Reasoning

Table 8 reports the HotpotQA multi-hop subset—expanded from 20 to 100 questions from the official development set and re-run under the fair generative protocol—and Table 9 breaks the Full Benchmark down by question type. Two things stand out. First, the multi-hop result is metric-dependent. HotpotQA references are short English factoids (for example, the name of an effect), so ROUGE-L penalizes TreeRAG’s longer generated answers and the flat baselines lead lexically (BM25 0.122 vs. TreeRAG-DFS 0.065); but on LLM-Judge, which rates answer quality rather than lexical overlap, TreeRAG-DFS is the best system (0.649), beating BM25 by +0.075 ($p = 0.008$) and RAPTOR by +0.064 ($p = 0.032$) with adequate power, while the Dense and FlatRAG comparisons remain positive but underpowered. The earlier $n = 20$ evaluation had instead shown TreeRAG winning ROUGE-L; the $n = 100$ fair-protocol re-run corrects that signal as an artifact of offline extractive scoring (Section 5.7), and we therefore frame the multi-hop contribution around LLM-Judge rather than ROUGE-L. Second, the per-type breakdown shows the consistent pattern across our work: a flat baseline wins ROUGE-L in every category, but TreeRAG-Beam attains the highest LLM-Judge on multi-hop (0.818) and comparative (0.850) questions and is competitive on factual (0.822). The multi-hop advantage of TreeRAG-Beam over Dense Retrieval is statistically significant ($p = 0.044$), consistent with the hypothesis that beam traversal helps bridge evidence across sibling sections. We state plainly that the per-type cells are small-sample results ($n = 11$ multi-hop, $n = 8$ comparative) and report them as indicative rather than conclusive.

Table 8: HotpotQA multi-hop ($n = 100$), fair generative protocol. The original 20-question subset was expanded to 100 questions from the official development set and all six systems re-run. Flat baselines lead on ROUGE-L because longest-common-subsequence recall rewards terse extractive matches over the longer fluent answers a generator produces; on LLM-Judge, TreeRAG-DFS attains the highest score (0.649), significantly above BM25 ($p = 0.008$) and RAPTOR ($p = 0.032$).

System	ROUGE-L	LLM-Judge
BM25	0.122	0.574
Dense Retrieval	0.102	0.598
FlatRAG	0.098	0.614
RAPTOR	0.086	0.585
TreeRAG-DFS	0.065	0.649
TreeRAG-Beam	0.065	0.621

Table 9: Per-type breakdown (Full Benchmark, $n = 40$, fair protocol). A flat baseline wins ROUGE-L in every category, while TreeRAG-Beam attains the highest LLM-Judge on multi-hop and comparative questions (multi-hop margin over Dense Retrieval significant, $p = 0.044$).

Question type (n)	Best ROUGE-L (system)	TreeRAG-Beam LLM-Judge
Factual (21)	0.566 (FlatRAG)	0.822
Multi-hop (11)	0.233 (FlatRAG)	0.818

Comparative (8)	0.169 (BM25)	0.850
-----------------	--------------	-------

6 DISCUSSION

6.1 Why Structure Helps Citations, Not Lexical Overlap

Our results draw a sharp line between two kinds of benefit that hierarchical retrieval is often assumed to deliver together. The first-page-level provenance and a compact, structurally coherent context-is real and survives fair evaluation: TreeRAG-Beam cites the right sections most accurately (Table 7) and reaches the highest judged quality on an order of magnitude fewer context tokens (Table 6). These follow directly from the representation: each node carries a page range, and selecting a relevant subtree yields a small, on-topic context rather than a scattering of chunks. The second supposed benefit-higher lexical overlap with reference answers-does not survive: once every system generates fluent text with the same LLM, the section boundary confers no advantage in n-gram overlap, and a hybrid flat retriever (FlatRAG) actually scores higher on ROUGE-L. In short, structure buys verifiability and efficiency, not surface-string similarity, and earlier hierarchical-RAG results that claimed large ROUGE-L gains likely measured an evaluation artifact rather than a retrieval improvement.

6.2 DFS versus Beam Search

Under the fair protocol, Beam Search is the better default. It attains the highest LLM-Judge overall (0.827), the best citation F1 (0.757), and the only statistically significant per-type advantage we observe (multi-hop over Dense Retrieval, $p = 0.044$). DFS is more token-frugal still (9 vs. 14 tokens) but scores lower on quality and is roughly twice as slow because it expands more nodes through the LLM scorer. We therefore recommend Beam as the production default and reserve DFS for extreme context-budget constraints. A query-complexity classifier that routes between the two remains attractive future work.

6.3 Threats to Validity

Several caveats bound our conclusions. Sample size: the fair-protocol experiments use a 40-question sample of the Full Benchmark and a 100-question HotpotQA subset (expanded from the original 20), which limits statistical power; most pairwise differences are not significant at this scale, and the per-type cells ($n = 8-21$) are smaller still. Single generator and judge: all generative results use one local model (Llama 3.1 8B) for both answer generation and LLM-as-Judge, so judge scores could carry model-specific preferences; because every answer under comparison is produced by that same model, this is not a per-system self-preference, but corroborating the judgement with an independent model remains important. Citation metric: Citation F1 is computed against gold supporting sections and rewards systems that emit section/page identifiers; it measures provenance fidelity, not answer correctness. Generator scale: an 8B local model is weaker than hosted frontier models, and absolute scores would likely rise with a stronger generator, though we have no evidence the relative ordering would change. Larger-sample, multi-model replication is the primary item of future work.

6.4 Limitations

Beyond the evaluation caveats of Section 6.3, the system has three engineering limitations. LLM dependency: indexing requires an LLM call per document, and traversal issues several LLM calls for node scoring, which is the source of TreeRAG's higher query latency. Structural prompt brittleness: documents with inconsistent heading styles (for example scanned PDFs with OCR artifacts) may yield malformed index trees requiring manual correction. Learned scoring: we attempted learned node scoring via DSPy [15] with an open 70B model; optimization yielded no improvement (0.0% gain),

likely due to insufficient training signal (fewer than 100 labeled examples), and we exclude this component from the main system.

6.5 Broader Impact

Because TreeRAG preserves page-level provenance, it is well suited to high-stakes domains-medicine, law, and regulatory compliance-where unsupported claims carry real cost and every assertion must be traceable to a source span. The same property aids auditing and post-hoc verification. Conversely, the system inherits the biases of its underlying LLM during indexing and generation, and the lightweight hallucination detector reduces but does not eliminate unfaithful output; deployments in sensitive settings should retain a human-in-the-loop for final decisions.

7 CONCLUSION

We presented TreeRAG, a hierarchical document RAG system that indexes PDFs into page-referenced JSON trees using zero-shot LLM prompting, traverses them with DFS or Beam Search, compresses retrieved context, and validates generated answers with a lightweight five-signal hallucination detector. Evaluating all systems under a fair protocol in which only the retriever differs and a shared local LLM (Llama 3.1 8B) generates every answer, we found that TreeRAG-Beam produces the most accurate page-level citations (citation F1 = 0.757 vs. 0.000 for the abstractive RAPTOR baseline), the highest LLM-judged answer quality on multi-hop and comparative questions (0.818 and 0.850; multi-hop advantage over Dense Retrieval significant at $p = 0.044$), and competitive overall quality (highest LLM-Judge, 0.827) while feeding an order of magnitude fewer context tokens to the generator-at the cost of higher per-query latency. Just as importantly, we showed that offline extractive evaluation and lexical-overlap metrics reverse these conclusions and overstate flat retrieval, and we reported both regimes so that the discrepancy is explicit; we encourage future hierarchical-RAG work to adopt generator-controlled evaluation and to measure citation and efficiency directly. The system also provides domain-adaptive prompting, multilingual support, and a production-ready API with caching, rate limiting, and asynchronous task queuing. Future work will pursue (1) larger-sample, multi-model replication of the fair protocol; (2) Graph RAG integration [3] for inter-document relationship modeling; (3) multi-modal indexing for figures and tables; and (4) active learning for traversal-policy optimization from user-feedback signals.

GenAI Usage Disclosure

Generative AI is a core component of the described system (LLM-driven tree indexing, answer generation, and LLM-as-judge evaluation), as detailed in Sections 3-5. In preparing this manuscript, generative AI tools were also used for language editing and reference formatting; all technical content, experimental design, analysis, and claims were produced and verified by the authors.

REFERENCES

- [1] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. In Proc. ICLR 2024. arXiv:2310.11511.
- [2] Tom B. Brown et al. 2020. Language Models are Few-Shot Learners. In Advances in Neural Information Processing Systems 33 (NeurIPS 2020), 1877-1901. arXiv:2005.14165.
- [3] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. 2025. From Local to Global: A GraphRAG Approach to Query-Focused Summarization. arXiv:2404.16130.
- [4] Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. 2023. RAGAS: Automated Evaluation of Retrieval Augmented Generation. arXiv:2309.15217.
- [5] Thibault Formal, Benjamin Piwowarski, and Stephane Clinchant. 2021. SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking. In Proc. SIGIR 2021, 2288-2292. <https://doi.org/10.1145/3404835.3463098>.

- [6] Thibault Formal, Stephane Clinchant, Herve Dejean, and Carlos Lassance. 2024. SPLATE: Sparse Late Interaction Retrieval. In Proc. SIGIR 2024. arXiv:2404.13950.
- [7] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2024. Retrieval-Augmented Generation for Large Language Models: A Survey. arXiv:2312.10997.
- [8] Shailja Gupta, Rajesh Ranjan, and Surya Narayan Singh. 2024. A Comprehensive Survey of Retrieval-Augmented Generation (RAG): Evolution, Current Landscape and Future Directions. arXiv:2410.12837.
- [9] Lei Huang et al. 2025. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions. ACM Trans. Inf. Syst. 43, 2, Article 42. <https://doi.org/10.1145/3703155>.
- [10] Haoyu Huang, Yongfeng Huang, Junjie Yang, Zhenyu Pan, Yongqiang Chen, Kaili Ma, Hongzhi Chen, and James Cheng. 2025. Retrieval-Augmented Generation with Hierarchical Knowledge. arXiv:2503.10150.
- [11] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. LLMingua: Compressing Prompts for Accelerated Inference of Large Language Models. In Proc. EMNLP 2023, 13358-13376. arXiv:2310.05736.
- [12] Jeff Johnson, Matthijs Douze, and Herve Jegou. 2019. Billion-scale Similarity Search with GPUs. IEEE Trans. Big Data 7, 3, 535-547. arXiv:1702.08734.
- [13] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In Proc. EMNLP 2020, 6769-6781.
- [14] Omar Khattab and Matei Zaharia. 2020. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In Proc. SIGIR 2020, 39-48. <https://doi.org/10.1145/3397271.3401075>.
- [15] Omar Khattab, Arnab Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. In Proc. ICLR 2024. arXiv:2310.03714.
- [16] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. 2022. Large Language Models are Zero-Shot Reasoners. In Advances in Neural Information Processing Systems 35 (NeurIPS 2022). arXiv:2205.11916.
- [17] Wojciech Kryscinski, Bryan McCann, Caiming Xiong, and Richard Socher. 2020. Evaluating the Factual Consistency of Abstractive Text Summarization. In Proc. EMNLP 2020, 9332-9346. arXiv:1910.12840.
- [18] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In Proc. SOSP 2023, 611-626. <https://doi.org/10.1145/3600006.3613165>.
- [19] Patrick Lewis et al. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In Advances in Neural Information Processing Systems 33 (NeurIPS 2020), 9459-9474. arXiv:2005.11401.
- [20] Chin-Yew Lin. 2004. ROUGE: A Package for Automatic Evaluation of Summaries. In Proc. ACL Workshop on Text Summarization Branches Out, 74-81.
- [21] Jerry Liu. 2022. LlamaIndex. GitHub. https://github.com/run-llama/llama_index.
- [22] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranajpe, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the Middle: How Language Models Use Long Contexts. Trans. Assoc. Comput. Linguist. 12, 157-173. arXiv:2307.03172.
- [23] Xueguang Ma, Ruqing Sun, Ronak Pradeep, and Jimmy Lin. 2021. A Replication Study of Dense Passage Retrieval for Open-Domain Question Answering. arXiv:2104.05740.
- [24] Ben Malin, Tatiana Kalganova, and Nikolaos Boulgouris. 2025. A Review of Faithfulness Metrics for Hallucination Assessment in Large Language Models. arXiv:2501.00269.
- [25] Sewon Min, Kalpesh Krishna, Xinxin Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. 2023. FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. In Proc. EMNLP 2023. arXiv:2305.14251.
- [26] Feng Nan, Ramesh Nallapati, Zhiguo Wang, Cicero Nogueira dos Santos, Henghui Zhu, Dejiao Zhang, Kathleen McKeown, and Bing Xiang. 2021. Entity-level Factual Consistency of Abstractive Text Summarization. In Proc. EACL 2021, 2727-2733. arXiv:2102.09130.
- [27] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In Proc. EMNLP 2019, 3982-3992. arXiv:1908.10084.
- [28] Stephen Robertson and Hugo Zaragoza. 2009. The Probabilistic Relevance Framework: BM25 and Beyond. Found. Trends Inf. Retr. 3, 4, 333-389.
- [29] Keshav Santhanam, Omar Khattab, Jon Saad-Falcon, Christopher Potts, and Matei Zaharia. 2022. ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction. In Proc. NAACL-HLT 2022, 3715-3734. arXiv:2112.01488.
- [30] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. 2024. RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. In Proc. ICLR 2024. arXiv:2401.18059.
- [31] Kurt Shuster, Spencer Poff, Moya Chen, Douwe Kiela, and Jason Weston. 2021. Retrieval Augmentation Reduces Hallucination in Conversation. In Findings of EMNLP 2021, 3784-3803. arXiv:2104.07567.
- [32] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. In Proc. ACL 2023, 10014-10037. arXiv:2212.10509.
- [33] Vectify AI. 2023. PageIndex: Reasoning-Based, Vectorless RAG via Document Tree Search. GitHub. <https://github.com/VectifyAI/PageIndex>.
- [34] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In Advances in Neural Information Processing Systems 35 (NeurIPS 2022). arXiv:2201.11903.
- [35] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A

Dataset for Diverse, Explainable Multi-hop Question Answering. In Proc. EMNLP 2018, 2369-2380. arXiv:1809.09600.

[36] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. 2020. BERTScore: Evaluating Text Generation with BERT. In Proc. ICLR 2020. arXiv:1904.09675.

[37] Xiaoming Zhang, Ming Wang, Xiaocui Yang, Daling Wang, Shi Feng, and Yifei Zhang. 2024. Hierarchical Retrieval-Augmented Generation Model with Rethink for Multi-hop Question Answering. arXiv:2408.11875.

[38] Lianmin Zheng et al. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In Advances in Neural Information Processing Systems 36 (NeurIPS 2023), Datasets and Benchmarks Track. arXiv:2306.05685.